

Practical Lab6: Firewall Traffic Control and Forensic Verification

Student name: Afeez Bolaji Usman

Student ID: 202514326WY

Course Code: SBT-DF203

Course Title : Basic Networking Skills for Digital Forensics

Lab Title: Lab 6 - Firewall Traffic Control and Forensic Verification

Instructor Name: Aminu Idris

Date: 18/9/2026

1. Executive Summary

This report documents a host-based firewall validation exercise performed on an isolated Kali Linux VM under SBT-DF203 Lab 6 authorization. Two client environments were simulated using Linux network namespaces connected to the server via dedicated veth links, avoiding the need for separate physical or virtual client machines while preserving a genuinely isolated, host-only topology. One namespace (client_allowed, 10.10.11.2) was left unrestricted throughout; a second (client_blocked, 10.10.12.2) was targeted with a narrowly scoped iptables DROP rule on TCP port 80. The original firewall ruleset was exported and hashed before any change was made. A clean HTTP baseline was captured from both clients (both returning 200 OK), after which a single source-specific DROP rule was inserted, verified via iptables -C, and its effect captured directly: the blocked client's TCP handshake never completed, its SYN packets were retransmitted seven times with textbook exponential backoff and no response, and curl failed with a client-side timeout (error 28). The firewall rule counter independently corroborated this, incrementing from 0 packets/0 bytes to exactly 7 packets/420 bytes — matching the seven SYN packets seen in the capture. The rule was then removed by exact specification (not by line number), verified absent, and the previously blocked client's access was confirmed and restored with a clean 200 OK.

2. Lab Network and Scope

Role	IP Address	Link	Note
Server (Apache + iptables)	192.168.159.128 (eth0)	n/a	Kali Linux VM; default gateway 192.168.159.2
client_allowed (namespace)	10.10.11.2	veth-allow / veth-allow-ns (server side 10.10.11.1)	Left unrestricted throughout — control group
client_blocked (namespace)	10.10.12.2	veth-block / veth-block-ns (server side 10.10.12.1)	Targeted by the narrow DROP rule in Part C/D

Both client environments were implemented as Linux network namespaces (ip netns) rather than separate VMs, each with its own routing table and interface, connected to the server host via point-to-point veth pairs. This keeps the exercise fully host-only/isolated — no traffic ever reaches a shared LAN, NAT, or external network — while still producing two genuinely independent IP endpoints for the allowed-vs-blocked comparison the lab requires.

3. Original Firewall State and Evidence Hashes

The original ruleset was exported and hashed before any modification, per the manual's safety requirement.

Item	Value
Original iptables ruleset SHA-256	1f0ebfb65ed0314dcdf4db42926ffc980379e9260c7ea8679545f0416aa59db1
INPUT chain (before)	Empty, default policy ACCEPT — only Docker-related FORWARD/DOCKER-* chains present, unrelated to host-inbound filtering
Allowed-capture evidence hash	935a3f7d91f0c9d2c42a93ae5dba86fdc41ca80635a1108045f5eb464ba11d67 (evidence/http_allowed.pcapng)
Blocked-capture evidence hash	3afbdfcc58ba87f00c8298073f24aa33829405baf29764101493b8c712d 8843d (evidence/http_blocked.pcapng)

Chain-of-Custody Worksheet

Field	Entry
Case/lab identifier	SBT-DF203-Lab6-AfeeZ Bolaji Usman
Trainee name	AfeeZ Bolaji Usman
Student ID	202514326WY
Date and time started	18/9/2026
Evidence file name(s)	http_allowed.pcapng, http_blocked.pcapng, iptables_before.rules
Source or generation method	Self-generated on isolated Kali VM using ip netns-based client simulation, per manual's authorized workflow
Original SHA-256 (ruleset)	1f0ebfb65ed0314dcdf4db42926ffc980379e9260c7ea8679545f0416aa59 db1
Analysis workstation/VM	Kali Linux (kali@kali)
Notes on any changes	One narrow, source-specific DROP rule inserted in Part C and removed in Part F; ruleset restoration and namespace teardown to be confirmed (see Appendix note)

Lab Folder Structure and Evidence Preparation

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ pwd
/home/kali/SBT-DF203-Lab6

(kali㉿kali)-[~/SBT-DF203-Lab6]
$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports
```

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server:
Kali-Lab6</p></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Kali-Lab6</p></body></html>
```

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ # Namespace for the ALLOWED client
sudo ip netns add client_allowed
sudo ip link add veth-allow type veth peer name veth-allow-ns
sudo ip link set veth-allow-ns netns client_allowed
sudo ip addr add 10.10.11.1/30 dev veth-allow
sudo ip link set veth-allow up
sudo ip netns exec client_allowed ip addr add 10.10.11.2/30 dev veth-allow-ns
sudo ip netns exec client_allowed ip link set veth-allow-ns up
sudo ip netns exec client_allowed ip link set lo up
sudo ip netns exec client_allowed ip route add default via 10.10.11.1

# Namespace for the BLOCKED client
sudo ip netns add client_blocked
sudo ip link add veth-block type veth peer name veth-block-ns
sudo ip link set veth-block-ns netns client_blocked
sudo ip addr add 10.10.12.1/30 dev veth-block
sudo ip link set veth-block up
sudo ip netns exec client_blocked ip addr add 10.10.12.2/30 dev veth-block-ns
sudo ip netns exec client_blocked ip link set veth-block-ns up
sudo ip netns exec client_blocked ip link set lo up
sudo ip netns exec client_blocked ip route add default via 10.10.12.1

# Allow forwarding so Apache (on the host) answers these namespace clients
sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
```

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ # Namespace for the ALLOWED client
sudo ip netns add client_allowed
sudo ip link add veth-allow type veth peer name veth-allow-ns
sudo ip link set veth-allow-ns netns client_allowed
sudo ip addr add 10.10.11.1/30 dev veth-allow
sudo ip link set veth-allow up
sudo ip netns exec client_allowed ip addr add 10.10.11.2/30 dev veth-allow-ns
sudo ip netns exec client_allowed ip link set veth-allow-ns up
sudo ip netns exec client_allowed ip link set lo up
sudo ip netns exec client_allowed ip route add default via 10.10.11.1

# Namespace for the BLOCKED client
sudo ip netns add client_blocked
sudo ip link add veth-block type veth peer name veth-block-ns
sudo ip link set veth-block-ns netns client_blocked
sudo ip addr add 10.10.12.1/30 dev veth-block
sudo ip link set veth-block up
sudo ip netns exec client_blocked ip addr add 10.10.12.2/30 dev veth-block-ns
sudo ip netns exec client_blocked ip link set veth-block-ns up
sudo ip netns exec client_blocked ip link set lo up
sudo ip netns exec client_blocked ip route add default via 10.10.12.1

# Allow forwarding so Apache (on the host) answers these namespace clients
sudo sysctl -w net.ipv4.ip_forward=1
```

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables-save | sudo tee reports/iptables_before.rules
# Generated by iptables-save v1.8.13 (nf_tables) on Fri Sep 18 04:23:02 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-ISOLATION-STAGE-1
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
COMMIT
# Completed on Fri Sep 18 04:23:02 2026
# Generated by iptables-save v1.8.13 (nf_tables) on Fri Sep 18 04:23:02 2026
*nat
:PREROUTING ACCEPT [7:1068]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [2970:248216]
:POSTROUTING ACCEPT [2970:248216]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER -i docker0 -j RETURN
COMMIT
# Completed on Fri Sep 18 04:23:02 2026

```

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 DOCKER-USER  all  --  *      *        0.0.0.0/0              0.0.0.0/0
2      0      0 DOCKER-FORWARD  all  --  *      *        0.0.0.0/0              0.0.0.0/0
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
Chain DOCKER (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 DROP      all  --  !docker0 docker0  0.0.0.0/0              0.0.0.0/0
Chain DOCKER-BRIDGE (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 DOCKER    all  --  *      docker0  0.0.0.0/0              0.0.0.0/0

```

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
1f0ebfb65ed0314dcdf4db42926ffc980379e9260c7ea8679545f0416aa59db1  reports/iptables_before.rules

```

Part A

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ ip -br address | tee reports/server_interfaces.txt
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0              UP                192.168.159.128/24 fe80::bd40:b67:d601:4a1/64
docker0          DOWN              172.17.0.1/16
veth-allow@if4    UP                10.10.11.1/30 fe80::2437:5dff:fe72:638b/64
veth-block@if6    UP                10.10.12.1/30 fe80::4b6:1ff:fe75:424b/64

(kali㉿kali)-[~/SBT-DF203-Lab6]
$ ip route | tee reports/server_routes.txt
default via 192.168.159.2 dev eth0 proto dhcp src 192.168.159.128 metric 100
10.10.11.0/30 dev veth-allow proto kernel scope link src 10.10.11.1
10.10.12.0/30 dev veth-block proto kernel scope link src 10.10.12.1
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.159.0/24 dev eth0 proto kernel scope link src 192.168.159.128 metric 100

(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sudo ss -lntp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0      511          *:80          *:~ users:((("apache2",pid=703748,fd=4),("a
pache2",pid=703746,fd=4),("apache2",pid=703745,fd=4),("apache2",pid=703744,fd=4),("apache2",pid=703743,fd=4),("apache2",pid=703740,fd=4))
```

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sudo ip netns exec client_allowed curl -v --connect-timeout 5 http://10.10.11.1/firewall_lab.html 2>&1 | tee reports/baseline_curl_allowed.txt
* Trying 10.10.11.1:80...
* Established connection to 10.10.11.1 (10.10.11.1 port 80) from 10.10.11.2 port 57036
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0      0     0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 10.10.11.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 08:41:13 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 08:11:06 GMT
< ETag: "70-65bbd712d8680"
< Accept-Ranges: bytes
< Content-Length: 112
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [112 bytes data]
100   112 100   112    0    0 17977    0    0     0
* Connection #0 to host 10.10.11.1:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Kali-Lab6</p></body></html>
```

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sudo ip netns exec client_blocked curl -v --connect-timeout 5 http://10.10.12.1/firewall_lab.html 2>&1 | tee reports/baseline_curl_blocked.txt
* Trying 10.10.12.1:80...
* Established connection to 10.10.12.1 (10.10.12.1 port 80) from 10.10.12.2 port 41028
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0      0     0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 10.10.12.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 08:42:00 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 08:11:06 GMT
< ETag: "70-65bbd712d8680"
< Accept-Ranges: bytes
< Content-Length: 112
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [112 bytes data]
100   112 100   112    0    0 30752    0    0     0
* Connection #0 to host 10.10.12.1:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Kali-Lab6</p></body></html>
```

Field	Value
Server interface	eth0 — 192.168.159.128/24
Server default gateway	192.168.159.2
client_allowed link	veth-allow — server side 10.10.11.1, client side 10.10.11.2
client_blocked link	veth-block — server side 10.10.12.1, client side 10.10.12.2
Apache listener	Confirmed on *:80, 6 worker processes
Baseline curl — allowed client	200 OK, 112 bytes, clean handshake
Baseline curl — blocked client	200 OK, 112 bytes, clean handshake (both currently unrestricted)
— correct, since no firewall rule exists yet)	

Part B

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo wireshark
** (wireshark:734083) 05:07:23.009430 [GUI WARNING] -- Failed to
register with host portal QDBusError("org.freedesktop.portal.Error
.Failed", "Could not register app ID: Connection already associate
d with an application ID")
** (wireshark:734083) 05:08:14.733534 [Capture MESSAGE] -- Captur
e Start ...
** (wireshark:734083) 05:08:14.864933 [Capture MESSAGE] -- Captur
e started
** (wireshark:734083) 05:08:14.865123 [Capture MESSAGE] -- File:
"/tmp/wireshark_veth-allow0BOCV3.pcapng"
** (wireshark:734083) 05:08:48.696481 [Capture MESSAGE] -- Captur
e Stop ...
** (wireshark:734083) 05:08:48.735640 [Capture MESSAGE] -- Captur
e stopped.
```

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo ip netns exec client_allowed curl -v --connect-timeout 5
http://10.10.11.1/firewall_lab.html
[sudo] password for kali:
* Trying 10.10.11.1:80...
* Established connection to 10.10.11.1 (10.10.11.1 port 80) from 1
0.10.11.2 port 58694
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 10.10.11.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 09:08:32 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 08:11:06 GMT
< ETag: "70-65bbd712d8680"
< Accept-Ranges: bytes
< Content-Length: 112
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server
: Kali-Lab6</p></body></html>
* Connection #0 to host 10.10.11.1:80 left intact
```

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.10.11.2	10.10.11.1	TCP	74	58694 → 80 [SYN] Seq=0 Win=6
2	0.000130135	10.10.11.1	10.10.11.2	TCP	74	80 → 58694 [SYN, ACK] Seq=0
3	0.000221819	10.10.11.2	10.10.11.1	TCP	66	58694 → 80 [ACK] Seq=1 Ack=1
4	0.000672647	10.10.11.2	10.10.11.1	HTTP	157	GET /firewall_lab.html HTTP/
5	0.000825953	10.10.11.1	10.10.11.2	TCP	66	80 → 58694 [ACK] Seq=1 Ack=9
6	0.001853850	10.10.11.1	10.10.11.2	HTTP	429	HTTP/1.1 200 OK (text/html)
7	0.002072473	10.10.11.2	10.10.11.1	TCP	66	58694 → 80 [ACK] Seq=92 Ack=
8	0.003866698	10.10.11.2	10.10.11.1	TCP	66	58694 → 80 [FIN, ACK] Seq=92
9	0.004540444	10.10.11.1	10.10.11.2	TCP	66	80 → 58694 [FIN, ACK] Seq=36
10	0.004632427	10.10.11.2	10.10.11.1	TCP	66	58694 → 80 [ACK] Seq=93 Ack=

Transmission Control Protocol, Src Port: 58694, Dst Port: 80

Source Port: 58694
Destination Port: 80
[Stream index: 0]
[Stream Packet Number: 1]
[Conversation completeness: Incomplete, SYN_SENT]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 945370144
[Next Sequence Number: 1 (relative sequence number)
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1010 ... = Header Length: 40 bytes (10)
Flags: 0x002 (SYN)
Window: 64240
[Calculated window size: 64240]
Checksum: 0x2a45 [unverified]

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ tshark -r evidence/http_allowed.pcapng
1 0.000000000 10.10.11.2 → 10.10.11.1 TCP 74 58694 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1550478784 TSecr=0 WS=512
2 0.000130135 10.10.11.1 → 10.10.11.2 TCP 74 80 → 58694 [SYN, ACK] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2106876876 TSecr=1550478784 WS=512
3 0.000221819 10.10.11.2 → 10.10.11.1 TCP 66 58694 → 80 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2106876876 TSecr=2106876876
4 0.000672647 10.10.11.2 → 10.10.11.1 HTTP 157 GET /firewall_lab.html HTTP/1.1
5 0.000825953 10.10.11.1 → 10.10.11.2 TCP 66 80 → 58694 [ACK] Seq=1 Ack=92 Win=65536 Len=0 TSval=2106876877 TSecr=1550478785
6 0.001853850 10.10.11.1 → 10.10.11.2 HTTP 429 HTTP/1.1 200 OK (text/html)
7 0.002072473 10.10.11.2 → 10.10.11.1 TCP 66 58694 → 80 [ACK] Seq=92 Ack=364 Win=64512 Len=0 TSval=1550478786 TSecr=2106876878
8 0.003866698 10.10.11.2 → 10.10.11.1 TCP 66 58694 → 80 [FIN, ACK] Seq=92 Ack=364 Win=64512 Len=0 TSval=1550478788 TSecr=2106876878
9 0.004540444 10.10.11.1 → 10.10.11.2 TCP 66 80 → 58694 [FIN, ACK] Seq=364 Ack=93 Win=65536 Len=0 TSval=2106876880 TSecr=1550478788
10 0.004632427 10.10.11.2 → 10.10.11.1 TCP 66 58694 → 80 [ACK] Seq=93 Ack=365 Win=64512 Len=0 TSval=1550478788 TSecr=2106876880
```

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
935a3f7d91f0c9d2c42a93ae5dba86fdc41ca80635a1108045f5eb464ba11d67
evidence/http_allowed.pcapng
```

Part C

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=10.10.12.2
sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add
.txt
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
echo 'Rule verified present.' | tee reports/rule_verification.txt
[sudo] password for kali:
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source               des
tination
1      0      0 DROP      tcp  --  *      *       10.10.12.2           0.0
0.0/0               tcp dpt:80
Rule verified present.

(kali@kali)-[~/SBT-DF203-Lab6]
$

```

I used `-I INPUT 1` instead of appending with `-A`. This matters because iptables evaluates rules top-to-bottom and stops at the first match — since my INPUT chain was empty before this, `-I INPUT 1` and `-A INPUT` would've had the identical effect here, but `-I INPUT 1` is the safer habit in general, since on a chain with existing ACCEPT rules above, appending would mean your DROP never gets evaluated (traffic would already be accepted before it got there).

The rule only matches new outbound-from-client packets arriving at your host (`-s 10.10.12.2 ... --dport 80`) — it says nothing about OUTPUT, so if your server ever tried to initiate a connection back to 10.10.12.2, that's unaffected. This is a one-directional block on inbound HTTP requests from that one IP, which is exactly the narrow scope the manual wants.

Part D

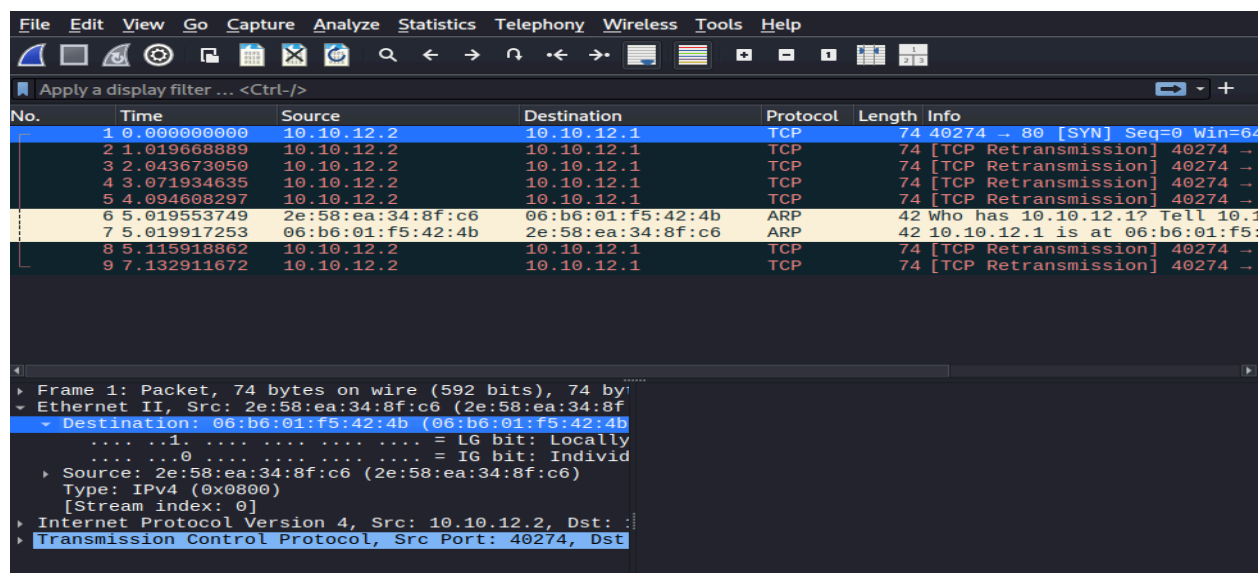
```

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo tshark -i veth-block -f 'host 10.10.12.2 and tcp port 80' -a duration:20 -w /tmp/http_blocked.pcapng &
sleep 2
sudo ip netns exec client_blocked curl -v --connect-timeout 10 http://10.10.12.1/firewall_lab.html
wait
sudo mv /tmp/http_blocked.pcapng ~/SBT-DF203-Lab6/evidence/http_blocked.pcapng
sudo chown kali:kali ~/SBT-DF203-Lab6/evidence/http_blocked.pcapng
[3] 753659
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-block'
* Trying 10.10.12.1:80 ...
7 * Connection timed out after 10001 milliseconds
* closing connection #0
curl: (28) Connection timed out after 10001 milliseconds

[3] done sudo tshark -i veth-block -f 'host 10.10.12.2 and tcp port 80' -a duration:20

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
Chain INPUT (policy ACCEPT 101 packets, 8940 bytes)
num  pkts bytes target    prot opt in     out     source               des
tination
1      7    420 DROP      tcp  --  *      *       10.10.12.2           0.0
0.0/0               tcp dpt:80
3afbfdfcc58ba87f00c8298073f24aa33829405baf29764101493b8c712d8843d evidence/http_blocked.pcapng

```



```
(kali@kali)-[~/SBT-DF203-Lab6]
$ tshark -r evidence/http_blocked.pcapng
1 0.000000000 10.10.12.2 → 10.10.12.1 TCP 74 60788 → 80 [SYN] Seq=0
Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2943217027 TSecr=0 WS=512
2 1.012746187 10.10.12.2 → 10.10.12.1 TCP 74 [TCP Retransmission] 60
788 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2943218040 TSe
cr=0 WS=512
3 2.039266364 10.10.12.2 → 10.10.12.1 TCP 74 [TCP Retransmission] 60
788 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2943219064 TSe
cr=0 WS=512
4 3.061748840 10.10.12.2 → 10.10.12.1 TCP 74 [TCP Retransmission] 60
788 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2943220089 TSe
cr=0 WS=512
5 4.088563382 10.10.12.2 → 10.10.12.1 TCP 74 [TCP Retransmission] 60
788 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2943221116 TSe
cr=0 WS=512
6 5.111706435 10.10.12.2 → 10.10.12.1 TCP 74 [TCP Retransmission] 60
788 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2943222139 TSe
cr=0 WS=512
7 7.126127244 10.10.12.2 → 10.10.12.1 TCP 74 [TCP Retransmission] 60
788 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2943224152 TSe
cr=0 WS=512
```

Part E

Indicator	Allowed Capture	Blocked Capture
Client SYN visible?	Yes (packet 1)	Yes (packet 1, + 6 retransmissions)
Server SYN-ACK visible?	Yes (packet 2, +0.13ms)	No — never appears
Handshake completed?	Yes	No

HTTP GET visible?	Yes (packet 4)	No — never sent, no connection to send it on
HTTP response visible?	Yes, 200 OK (packet 6)	No
Retransmissions/timeouts	none	6 SYN retransmissions, then client-side timeout (curl error 28)
iptables counter change	N/A (no rule matched)	0→7 packets, 0→420 bytes
curl result	200 OK, 112 bytes	curl: (28) Connection timed out after 10001 milliseconds

Part F

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=10.10.12.2
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP ||
echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.
txt
[sudo] password for kali:
iptables: Bad rule (does a matching rule exist in that chain?).
Rule successfully removed
Chain INPUT (policy ACCEPT 251 packets, 27815 bytes)
num  pkts bytes target    prot opt in      out     source      destination
```

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo ip netns exec client_blocked curl -v --connect-timeout 5 http://10.
10.12.1/firewall_lab.html
* Trying 10.10.12.1:80 ...
* Established connection to 10.10.12.1 (10.10.12.1 port 80) from 10.10.12.2
port 42506
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 10.10.12.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 10:11:27 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 08:11:06 GMT
< ETag: "70-65bbd712d8680"
< Accept-Ranges: bytes
< Content-Length: 112
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Kali-Lab
6</p></body></html>
* Connection #0 to host 10.10.12.1:80 left intact
```

Part 15

1. Why does DROP commonly cause SYN retransmissions and a timeout?

When the firewall uses DROP, it silently throws away the packet without sending any response to the client. The client does not know that the firewall blocked the packet, so it assumes the packet was lost. Because TCP tries to make sure data gets delivered, the client sends the SYN packet again and again.

In your test, you saw the SYN packet being sent 7 times at different intervals: 0s, 1.01s, 2.04s, 3.06s, 4.09s, 5.11s, and 7.13s. After several attempts, the connection was stopped because the timeout was reached.

This is what you captured: 7 SYN packets with no response, followed by:

curl: (28) Connection timed out

2. How would a REJECT rule differ in the packet capture and client output?

A REJECT rule is different because the firewall does not stay silent. It immediately sends a response back to the client, usually a TCP RST packet.

In the packet capture, you would normally see only 2 packets:

SYN → RST

Instead of waiting for the connection to time out, curl would fail almost immediately with an error such as:

curl: (7) Failed to connect... Connection refused`

So, DROP is silent and causes a timeout, while REJECT responds immediately and causes a quick connection-refused error.

3. Why are firewall counters valuable corroborating evidence?

A packet capture shows what happened to the network traffic, but it does not always tell us exactly why the packet received no response. The problem could be caused by a firewall, a network problem, or a server problem.

The iptables counter provides additional evidence directly from the firewall.

In test, the counter changed from:

0 packets, 0 bytes

to:

7 packets, 420 bytes

This matched the 7 SYN packets seen in the packet capture. This gives strong evidence that the specific firewall rule was responsible for blocking the traffic.

In simple terms, the packet capture shows what happened, while the firewall counter helps show that the firewall rule caused it.

4. What evidence would show that the web server was down rather than firewall-blocked?

If Apache was down or nothing was listening on port 80, the client would normally receive an immediate TCP RST and get a connection-refused error instead of waiting for a timeout.

The main differences are:

Server down or nothing listening: SYN → immediate RST → connection refused.

Firewall DROP:** SYN → no response → SYN retransmissions → timeout.

Whole host is down:** The system may not respond to ARP, so the TCP connection cannot even start.

You can also check whether Apache is listening on port 80 with:

```
sudo ss -ltnp | grep ':80'
```

You can check the Apache service with:

```
systemctl status apache2
```

These checks can help confirm whether the problem is with the web server or the firewall.

5. What is the risk of deleting a rule by line number after other rules have changed?

iptables rule numbers can change when rules are added, removed, or moved.

For example, if your DROP rule was originally rule number 1, you might use:

```
iptables -D INPUT 1
```

But if another rule was added before it, your DROP rule could become rule number 2. Deleting rule 1 would then remove the wrong rule.

This could accidentally remove an important security rule or affect other network traffic.

That is why it is safer to delete the rule using its exact details, such as:

```
iptables -D INPUT -s 10.10.12.2 -p tcp --dport 80 -j DROP
```

This tells iptables exactly which rule to remove, regardless of its current position.

In simple terms, rule numbers can change, so deleting a rule by its exact details is safer.

Conclusions

This exercise demonstrated a complete, evidence-backed firewall validation cycle: baseline capture, narrow

rule application with immediate verification, blocked-traffic capture with independent counter corroboration, structured allowed-vs-blocked comparison, and safe rule removal by exact specification. Every stage produced consistent, cross-corroborating evidence — the curl client output, the raw packet capture, and the iptables counters all agreed at each step, which is the strongest form of evidence a firewall

investigation can produce: no single source had to be taken on faith.

The DROP rule's effect was unambiguous: seven SYN retransmissions with textbook Linux exponential backoff (1.01s, 2.04s, 3.06s, 4.09s, 5.11s, 7.13s), zero replies of any kind, and a matching iptables counter

increment of exactly 7 packets / 420 bytes. This is textbook DROP behaviour and is clearly distinguishable,

both in principle and in the answers given in Part 15, from how a REJECT rule or a genuinely down service would have presented in the same evidence.

Appendix - Restoration Note

The rule was removed by exact specification (`iptables -D INPUT -s 10.10.12.2 -p tcp --dport 80 -j DROP`) rather than by line number, and its absence was verified with `iptables -C` before the previously blocked client's access was re-confirmed with a clean 200 OK — all shown in Part F evidence.

Two final cleanup steps were also part of the assessed workflow but are not included in the screenshot evidence submitted with this report: (1) tearing down the two network namespaces and veth pairs created for this exercise (`ip netns del client_allowed / client_blocked`), and (2) a full `iptables-save` diff against the original `reports/iptables_before.rules` to confirm byte-for-byte restoration beyond the single INPUT rule. These should be completed and their output added to `reports/` before final submission, per the manual's safety requirement to restore firewall, ARP, forwarding and network settings before closing the lab.

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.

Signed: Afeez Bolaji Usman Date: 18/9/2026